4.4.2. Having the agent interact with the environment

The agent consumes the state and takes an action, *a*, probabilistically. More specifically, the state is input to the policy network, which then produces the probability distribution over the actions $P(A|\theta,S_t)$ given its current parameters and the state. Note, the capital *A* refers to the set of all possible actions given the state, whereas the lowercase *a* generally refers to a particular action.

The policy network might return a discrete probability distribution in the form of a vector, such as [0.25, 0.75] for our two possible actions in CartPole. This means the policy network predicts that action 0 is the best with 25% probability, and action 1 is the best with 75% probability (or confidence). We call this array `pred`.

Listing 4.5. Using the policy network to sample an action

```
pred = model(torch.from_numpy(state1).float())                    1
action = np.random.choice(np.array([0,1]), p=pred.data.numpy())   2
state2, reward, done, info = env.step(action)                     3
```

- ***1*** Calls the policy network model to produce predicted action probabilities
- ***2*** Samples an action from the probability distribution produced by the policy network
- ***3*** Takes the action and receives the new state and reward. The info variable is produced by the environment but is irrelevant.

The environment responds to the action by producing a new state, s_2, and a reward, r_2. We store those into two arrays (a `states` array and an `actions` array) for when we need to update our model after the episode ends. We then plug the new state into our model, get a new state and reward, store those, and repeat until the episode ends (the pole falls over and the game is finished).

4.4.3. Training the model